

Project: Linear Regression Model on Housing Dataset

Dataset used: USA_Housing.csv attached below.



USA_Housing.csv

Connecting Dataset:

The screenshot shows the Tableau interface with the following components:

- Connections:** A list of connections including 'USA_Housing' (Text File).
- Files:** A list of files including 'model_results.csv', 'model_results_FINAL.csv', and 'USA_Housing.csv'.
- Main Workspace:** Displays the 'USA_Housing' dataset with a preview table showing 7 fields and 5000 rows. The preview table includes columns for 'Avg. Area Income', 'Avg. Area House Age', 'Avg. Area Number of Rooms', and 'Avg. Area Population'.
- Bottom Navigation Bar:** Includes tabs for 'Data Source', 'Sheet 1', 'Sheet 2', 'Sheet 3', 'Sheet 4', and 'Dashboard 1'.

Installing TabPy:

-m python pip install --upgrade pip

Pip install tabpy

```
Do you wish to proceed without authentication? (y/N): y
2023-10-27,22:46:05 [INFO] (app.py:app:492): Password file is not specified: Authentication is not enabled
2023-10-27,22:46:05 [INFO] (app.py:app:411): Call context logging is disabled
2023-10-27,22:46:05 [INFO] (app.py:app:181): Initializing TabPy...
2023-10-27,22:46:05 [INFO] (callbacks.py:callbacks:43): Initializing TabPy Server...
2023-10-27,22:46:05 [INFO] (app.py:app:185): Done initializing TabPy.
2023-10-27,22:46:05 [INFO] (callbacks.py:callbacks:64): Initializing models...
2023-10-27,22:46:05 [INFO] (app.py:app:151): Web service listening on port 9004
2023-10-27,22:48:34 [INFO] (base_handler.py:base_handler:115): Authentication is not a required feature for API "v1"
2023-10-27,22:48:34 [INFO] (base_handler.py:base_handler:115): Authorization header not found
2023-10-27,22:48:34 [INFO] (web.py:web:2344): 200 GET /info (127.0.0.1) 7.57ms
```

Manage Analytics Extensions Connection

×

Select a connection type

Click a connection type below. [Learn more](#)



TabPy

Integrate Python into Tableau calculations using TabPy, Tableau's Python server.



Einstein Discovery

Augment your analysis with Einstein Discovery predictions.



RServe

Integrate R into Tableau calculations using RServe, an open source R server.



Analytics Extensions API

Extend Tableau calculations to dynamically include external analytics tools and platforms.

Close

Manage Analytics Extensions Connection

×



New TabPy Connection

☐ Require SSL

Hostname

Port

☐ Sign in with username and password

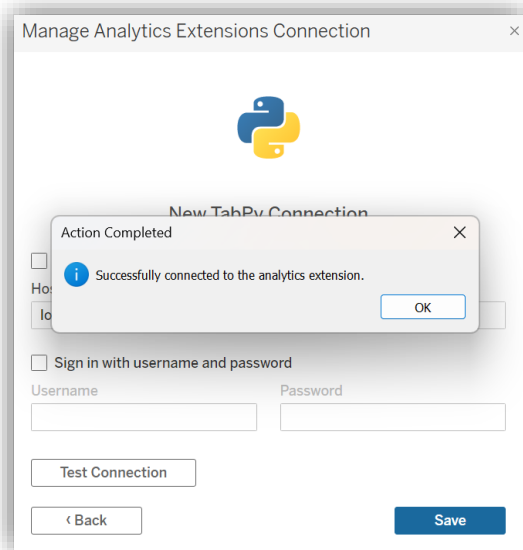
Username

Password

Test Connection

◀ Back

Save



SCRIPT for Models:

We have created two models:

1. Linear Regression Model (LR)
2. Multiple Linear Regression Model (MLR)

The goal was to see which model is predicting the Price of Houses better. As we know that MLR is trained on Multiple Variables of the dataset, MLR is able to predict more accurately than LR.

We have taken “Price” as our dependent variable.

Script of LR Model: (Trained on Avg. Area Income only)



```
SCRIPT_REAL(  
,  
import numpy as np  
from sklearn import linear_model  
  
lr = linear_model.LinearRegression()  
  
X = np.transpose(np.array([_arg1]))  
y = np.array(_arg2)  
  
lr.fit(X,y)  
return lr.predict(X).tolist()  
,  
AVG([Avg. Area Income]),
```

```
SUM([Price])
)
```

Script of MLR Model: (Trained on all the variables except Price)

MLR Predictions

Results are computed along Table (across).

```
SCRIPT_REAL(
',
import numpy as np
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
SCRIPT_REAL(string, expression, ...)
# Combine independent variables into a 2D array (each column corresponds to a feature)
X = np.column_stack((_arg1, _arg2, _arg3, _arg4, _arg5))

# Dependent variable
y = _arg6

lr.fit(X, y)

# Make predictions and return as a list
return lr.predict(X).tolist()
',
AVG([Avg. Area Income]),
AVG([Avg. Area House Age]),
```

Default Table Calculation

The calculation is valid.

5 Dependencies ▾

Apply

OK

```
SCRIPT_REAL(
',
```

```
import numpy as np
```

```
from sklearn.linear_model import LinearRegression
```

```
lr = LinearRegression()
```

```
# Combine independent variables into a 2D array (each column corresponds to a feature)
```

```
X = np.column_stack((_arg1, _arg2, _arg3, _arg4, _arg5))
```

```
# Dependent variable
```

```
y = _arg6
```

```
lr.fit(X, y)
```

```
# Make predictions and return as a list
```

```
return lr.predict(X).tolist()
```

```
,
```

```
AVG([Avg. Area Income]),
```

```
AVG([Avg. Area House Age]),
```

```
AVG([Avg. Area Number of Rooms]),
```

```
AVG([Avg. Area Number of Bedrooms]),
```

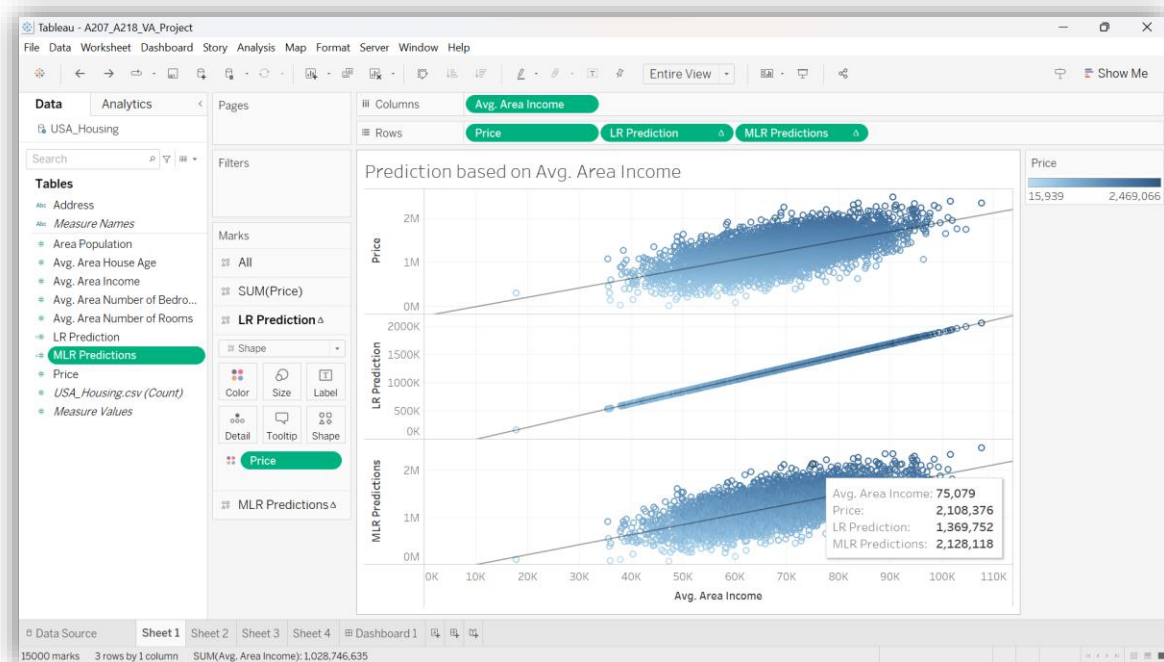
```
AVG([Area Population]),
```

```
SUM([Price])
```

```
)
```

VISUALIZATIONS:

Sheet 1:



In the above Screenshot you can clearly see that:

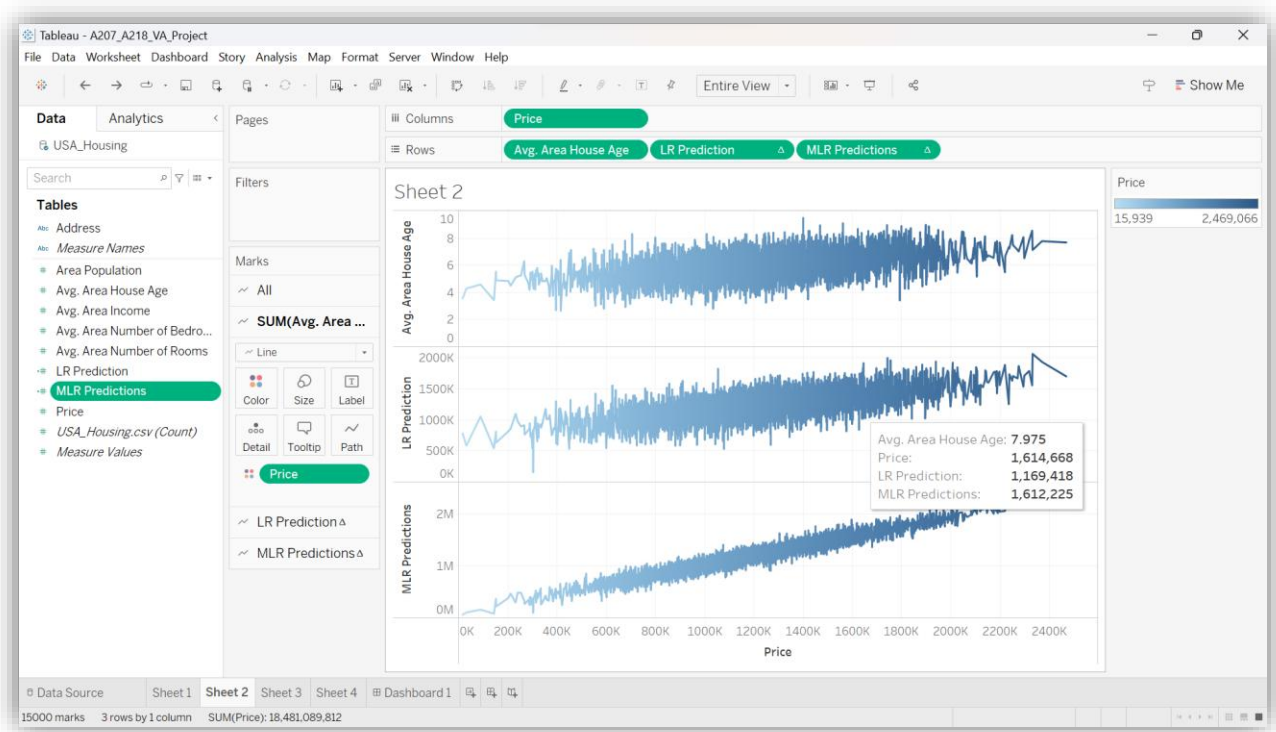
Actual Price is **2108376**

LR Prediction is **1369752** (which is very far from actual price)

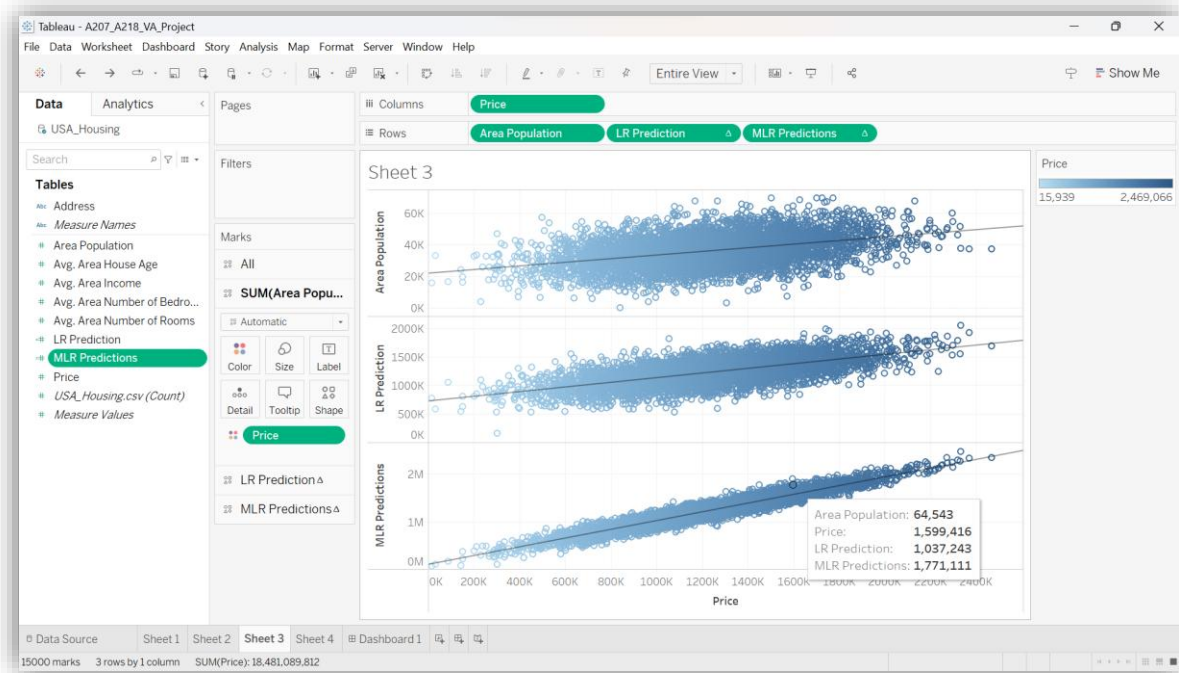
MLR Prediction is **2128118** (which is very near to the actual price)

So, MLR is able to predict more accurately.

Sheet 2:



Sheet 3:



Sheet 4:

